

সম্পূর্ণ গাইড · STEP BY STEP

Next.js শেখার পূর্ণাঙ্গ গাইড

অফিসিয়াল ডকুমেন্টেশন অনুসরণ করে, ধাপে ধাপে, সহজ বাংলায়

সংজ্ঞা → বাস্তব উপমা → ব্যবহারিক ক্ষেত্র → কোড উদাহরণ

 Routing

 Server Components

 Data Fetching

 Deployment

সূচিপত্র

TABLE OF CONTENTS

পর্ব ১ — শুরুর কথা — Getting Started

১.১	Next.js আসলে কী?	What is Next.js
১.২	প্রজেক্ট স্ট্রাকচার ও App Router	Project Structure

পর্ব ২ — রাউটিং — Routing (App Router)

২.১	ফাইল-বেজড রাউটিং (page.js)	Pages & File-based Routing
২.২	লেআউট (layout.js)	Layouts
২.৩	নেভিগেশন — Link ও useRouter	Linking and Navigating
২.৪	ডাইনামিক রাউট	Dynamic Routes
২.৫	রুট গ্রুপ	Route Groups

পর্ব ৩ — UI স্টেট ও এরর — Loading UI & Error Handling

৩.১	লোডিং UI ও স্ট্রিমিং	Loading UI & Streaming
৩.২	এরর হ্যান্ডলিং	Error Handling

পর্ব ৪ — কম্পোনেন্ট ও ডেটা — Components & Data Fetching

৪.১	সার্ভার কম্পোনেন্ট বনাম ক্লায়েন্ট কম্পোনেন্ট	Server vs Client Components
৪.২	ডেটা ফেচিং	Data Fetching
৪.৩	সার্ভার অ্যাকশন	Server Actions & Mutations
৪.৪	রুট হ্যান্ডলার (API রাউট)	Route Handlers

পর্ব ৫ — অপটিমাইজেশন — Optimization & Middleware

৫.১ মিডলওয়্যার

Middleware

৫.২ ইমেজ ও ফন্ট অপটিমাইজেশন

Image & Font Optimization

৫.৩ মেটাডেটা ও SEO

Metadata & SEO

পর্ব ৬ — রেন্ডারিং ও প্রোডাকশন — Rendering & Deployment

৬.১ স্ট্যাটিক বনাম ডাইনামিক রেন্ডারিং

Static vs Dynamic Rendering

৬.২ এনভায়রনমেন্ট ভেরিয়েবল

Environment Variables

৬.৩ ডেপ্লয়মেন্ট

Deployment

পর্ব ১ — শুরুৰ কথা

১.১ Next.js আসলে কী?

What is Next.js

■ সংজ্ঞা (Definition)

Next.js হলো React-এর উপরে বানানো একটি ফুল-স্ট্যাক ওয়েব ফ্রেমওয়ার্ক। শুধু React দিয়ে ব্রাউজারে UI বানানো যায়, কিন্তু রাউটিং, সার্ভার-সাইড রেন্ডারিং, ডেটা ফেচিং, ইমেজ অপটিমাইজেশন, API তৈরি — এসব আলাদা আলাদা টুল দিয়ে জোড়া দিতে হয়। Next.js এই সবগুলো জিনিস একসাথে, একটা সুশৃঙ্খল কনভেনশনের ভেতরে দিয়ে দেয়, যাতে ডেভেলপার শুধু প্রোডাক্ট বানানোতে মনোযোগ দিতে পারে।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

ধরুন আপনি একটা বাড়ি বানাবেন। শুধু React মানে হলো আপনার কাছে ইট, সিমেন্ট, রড আছে — কিন্তু বিদ্যুৎ লাইন, পানির লাইন, দরজা-জানালা ডিজাইন সব আপনাকে আলাদা করে ঠিক করতে হবে। আর Next.js হলো একটা রেডিমেড হাউজিং প্যাকেজ — কাঠামো, বিদ্যুৎ, পানি, নিরাপত্তা ব্যবস্থা সব আগে থেকেই ঠিক করা আছে, আপনি শুধু ভেতরের সাজসজ্জা (আপনার অ্যাপের লজিক) নিয়ে কাজ করবেন।

📍 কোথায় ব্যবহার করবেন (Where to Use)

যেকোনো প্রোডাকশন-গ্রেড ওয়েব অ্যাপ্লিকেশন বানাতে — যেমন ই-কমার্স সাইট, ব্লগ, ড্যাশবোর্ড, SaaS প্রোডাক্ট, কোম্পানির মার্কেটিং সাইট — যেখানে দ্রুত লোড টাইম, ভালো SEO আর স্কেলেবিলিটি দরকার, সেখানে Next.js প্রথম পছন্দ।

```
# একটি নতুন Next.js প্রজেক্ট তৈরি করা
npx create-next-app@latest my-app

cd my-app
npm run dev

# http://localhost:3000 এ অ্যাপ চালু হয়ে যাবে
```

Terminal



সার্ভার HTML রেসপন্স হিসেবে ব্রাউজারে ফেরত পাঠায়

চিত্র: Next.js আসলে কী? — ভিজুয়াল উপস্থাপন

১.২ প্রজেক্ট স্ট্রাকচার ও App Router

Project Structure

■ সংজ্ঞা (Definition)

নতুন Next.js প্রজেক্টে একটি app/ ফোল্ডার থাকে — একেই বলা হয় App Router। এই ফোল্ডারের প্রতিটি সাব-ফোল্ডার একটি করে রুট (route) সেগমেন্ট তৈরি করে। বিশেষ কিছু ফাইলনেম যেমন page.js, layout.js, loading.js — এদের প্রত্যেকটির নির্দিষ্ট ভূমিকা আছে এবং Next.js এগুলোকে স্বয়ংক্রিয়ভাবে চিনে নেয়।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

এটাকে একটা অফিস বিল্ডিং এর মতো ভাবুন। প্রতিটা ফ্লোর/রুম হলো একটা ফোল্ডার। প্রতিটা রুমের দরজায় একটা নির্দিষ্ট নেমপ্লেট থাকলে (যেমন page.js) বোঝা যায় ওটা ভিজিটরদের জন্য উন্মুক্ত রুম, আবার কিছু নেমপ্লেট (layout.js) মানে হলো এই ফ্লোরের কমন করিডোর বা রিসেপশন — যেটা প্রতিটা রুমে যাওয়ার আগে সবাইকে পার হতে হয়।

📍 কোথায় ব্যবহার করবেন (Where to Use)

প্রজেক্ট শুরু করার সময়ে ফাইল ও ফোল্ডার কীভাবে সাজাতে হবে তা বোঝার জন্য। যেকোনো নতুন ফিচার বা পেজ যোগ করার আগে জানতে হবে কোন ফাইল কোথায় বসবে।

```

my-app/
├── app/
│   ├── layout.js      # রুট লেআউট ( সব পেজে কমন)
│   ├── page.js        # হোমপেজ ( / )
│   ├── globals.css
│   ├── about/
│   │   └── page.js    # /about রুট
│   └── blog/
│       └── page.js    # /blog রুট
├── public/            # স্ট্যাটিক ফাইল ( ছবি, ফন্ট)
├── next.config.js
└── package.json
  
```

পর্ব ২ — রাউটিং

২.১ ফাইল-বেজড রাউটিং (page.js)

Pages & File-based Routing

📖 সংজ্ঞা (Definition)

Next.js-এ আলাদা করে রাউট কনফিগার করার দরকার হয় না। `app/` ফোল্ডারের ভেতরে যে ফোল্ডারে `page.js` ফাইল থাকে, সেই ফোল্ডারের পথ অনুযায়ী একটি URL রুট পাবলিকলি অ্যাক্সেসযোগ্য হয়ে যায়। একে বলে ফাইল-সিস্টেম বেজড রাউটিং।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

এটা অনেকটা লাইব্রেরির বই খুঁজে পাওয়ার মতো। প্রতিটা বইয়ের একটা নির্দিষ্ট শেলফ-নম্বর (ফোল্ডার পথ) থাকে, আর সেই শেলফে গেলেই বইটা (`page.js`) পাওয়া যায় — আলাদা কোনো ক্যাটালগ বা ম্যাপ বানিয়ে রাউট রেজিস্টার করতে হয় না, ফোল্ডার স্ট্রাকচারটাই ম্যাপ।

📍 কোথায় ব্যবহার করবেন (Where to Use)

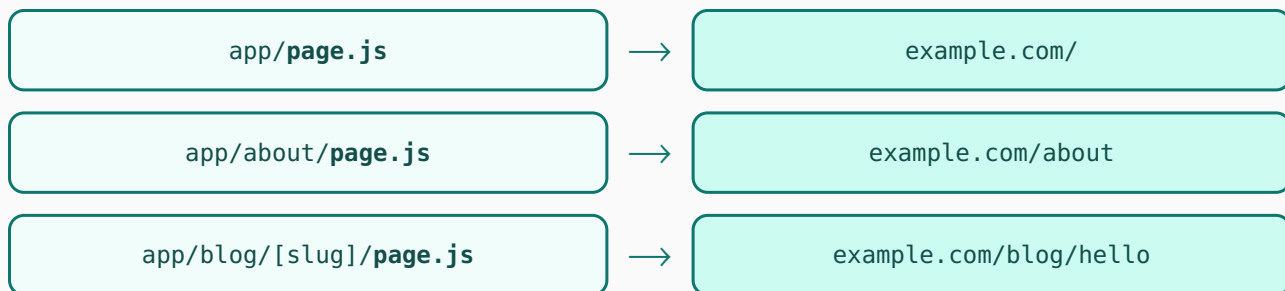
যখনই একটি নতুন পেজ/স্ক্রিন যোগ করতে হবে — যেমন `/about`, `/contact`, `/pricing` — তখন `app/` এর ভেতরে সেই নামে ফোল্ডার বানিয়ে একটা `page.js` রাখলেই কাজ শেষ।

```
JavaScript / JSX

// app/about/page.js
// এই ফাইলটি '/about' রুট তৈরি করে

export default function AboutPage() {
  return (
    <div>
      <h1>আমাদের সম্পর্কে</h1>
      <p>এই পেজটি /about URL-এ দেখা যাবে।</p>
    </div>
  );
}
```

app/ ফোল্ডার → URL ম্যাপিং



চিত্র: ফাইল-বেজড রাউটিং (page.js) — ভিজুয়াল উপস্থাপন

 সংজ্ঞা (Definition)

layout.js হলো এমন একটি UI যা একাধিক পেজের মধ্যে শেয়ার করা হয় এবং নেভিগেশনের সময় রি-রেভার হয় না, স্টেট ধরে রাখে। প্রতিটা লেআউট তার children prop-এর মাধ্যমে ভেতরের পেজ বা নেস্টেড লেআউট রেভার করে। রুট লেআউট (app/layout.js) বাধ্যতামূলক এবং এতে html ও body ট্যাগ থাকতে হয়।

 বাস্তব জীবনের উপমা (Real-life Analogy)

একটি রেস্টুরেন্টের কথা চিন্তা করুন — প্রতিটা টেবিলে খাবার (page content) আলাদা আলাদা আসে, কিন্তু রেস্টুরেন্টের দেয়াল, লাইটিং, মিউজিক (layout) সব টেবিলের জন্য একই থাকে। খাবার পরিবর্তন হলেও দেয়াল ভেঙে আবার বানাতে হয় না।

 কোথায় ব্যবহার করবেন (Where to Use)

নেভিগেশন বার, ফুটার, সাইডবার — যা প্রতিটা পেজে দেখাতে হবে কিন্তু বারবার লিখতে চান না, সেগুলোর জন্য layout.js ব্যবহার করবেন।

```
JavaScript / JSX

// app/layout.js
export default function RootLayout({ children }) {
  return (
    <html lang="bn">
      <body>
        <nav>এইটা সব পেজে দেখা যাবে (নেভবার)</nav>
        <main>{children}</main>
        <footer>ফুটার — সব পেজে কমন</footer>
      </body>
    </html>
  );
}
```

■ সংজ্ঞা (Definition)

সাধারণ ট্যাগ ব্যবহার করলে পুরো পেজ রিলোড হয়ে যায়। Next.js-এর কম্পোনেন্ট ক্লায়েন্ট-সাইড নেভিগেশন করে — পুরো পেজ রিলোড না করেই নতুন পেজের কন্টেন্ট বদলে দেয়, ফলে অ্যাপ অনেক দ্রুত ফিল হয়। প্রোগ্রাম্যাটিক্যালি (যেমন বাটন ক্লিকের পর) নেভিগেট করতে useRouter হুক ব্যবহার হয়।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

সাধারণ ট্যাগ হলো প্রতিবার নতুন করে দরজা ভেঙে ঘরে ঢোকার মতো, আর হলো একটা স্লাইডিং দরজার মতো — জুতা খোলার (পুরো পেজ লোড হওয়ার) দরকার নেই, শুধু স্লাইড করে ভেতরে চলে যাওয়া যায়, দ্রুত ও মসৃণ।

📍 কোথায় ব্যবহার করবেন (Where to Use)

নেভবার, বাটন, কার্ড — যেখানেই ইউজারকে এক পেজ থেকে অন্য পেজে পাঠাতে হবে সেখানে এর বদলে সবসময় ব্যবহার করা উচিত।

```
import Link from 'next/link';
import { useRouter } from 'next/navigation';

export default function Nav() {
  const router = useRouter();

  return (
    <div>
      <Link href="/blog">ব্লগে যান</Link>

      <button onClick={() => router.push('/dashboard')}>
        ড্যাশবোর্ডে যান
      </button>
    </div>
  );
}
```

■ সংজ্ঞা (Definition)

যখন রুটের একটি অংশ আগে থেকে জানা থাকে না — যেমন প্রতিটা ব্লগ পোস্টের জন্য আলাদা ID বা স্লাগ — তখন ফোল্ডারের নাম স্কয়ার ব্রাকেটে ([slug]) দিয়ে ডাইনামিক সেগমেন্ট তৈরি করা হয়। এই ভ্যালু params অবজেক্টের মাধ্যমে কম্পোনেন্টে পাওয়া যায়।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

এটা অনেকটা হোটেলের রুম নম্বরের মতো। হোটেলের ডিজাইন (page.js) একটাই — কিন্তু রুম নম্বর ১০১, ১০২, ১০৩ (params.slug) অনুযায়ী ভেতরের অতিথি (ডেটা) আলাদা হয়। একই টেমপ্লেট দিয়ে হাজারটা আলাদা রুম সার্ভ করা যায়।

📍 কোথায় ব্যবহার করবেন (Where to Use)

ব্লগ পোস্ট (/blog/[slug]), প্রোডাক্ট পেজ (/product/[id]), ইউজার প্রোফাইল (/user/[username]) — যেখানেই একই লেআউটে ভিন্ন ভিন্ন ডেটা দেখাতে হবে সেখানে ডাইনামিক রাউট ব্যবহার হয়।

```
JavaScript / JSX

// app/blog/[slug]/page.js

export default function BlogPost({ params }) {
  const { slug } = params;

  return (
    <div>
      <h1>ব্লগ পোস্ট: {slug}</h1>
      <p>এই পেজটি /blog/hello-world, /blog/nextjs-guide
      ইত্যাদি যেকোনো স্লাগের জন্য কাজ করবে।</p>
    </div>
  );
}
```

■ সংজ্ঞা (Definition)

রুট গ্রুপ তৈরি হয় ফোল্ডারের নাম বন্ধনীতে (folder) রেখে — এই নাম URL-এ যুক্ত হয় না, শুধু ফাইলগুলোকে সংগঠিত করার এবং আলাদা আলাদা লেআউট প্রয়োগ করার কাজে ব্যবহৃত হয়।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

এটা অফিসের ফাইল ক্যাবিনেটের মতো — ক্যাবিনেটের ড্রয়ারের গায়ে লেখা 'অ্যাকাউন্টস' বা 'HR' লেবেলটা শুধু আপনার নিজের গোছানোর সুবিধার জন্য, বাইরের কেউ ফাইল খুঁজতে গিয়ে সেই লেবেলের নাম উচ্চারণ করে না — সরাসরি ফাইলের নাম ধরেই খোঁজে।

📍 কোথায় ব্যবহার করবেন (Where to Use)

যখন (marketing) আর (shop) এর মতো দুই ভাগের পেজের জন্য আলাদা আলাদা লেআউট দরকার, কিন্তু URL-এ কোনো এক্সট্রা সেগমেন্ট (যেমন /marketing/about) দেখাতে চান না।

```
app/
├── (marketing)/
│   ├── layout.js      # মার্কেটিং পেজের আলাদা লেআউট
│   ├── about/page.js  # URL: /about (marketing শব্দটা নেই)
│   └── blog/page.js    # URL: /blog
└── (shop)/
    ├── layout.js      # শপ সেকশনের আলাদা লেআউট
    └── cart/page.js    # URL: /cart
```

Terminal

পর্ব ৩ — UI স্টেট ও এরর

৩.১ লোডিং UI ও স্ট্রিমিং

Loading UI & Streaming

■ সংজ্ঞা (Definition)

কোনো ফোল্ডারে loading.js ফাইল রাখলে, সেই সেক্সনের ডেটা লোড হওয়ার সময়ে Next.js স্বয়ংক্রিয়ভাবে একটি লোডিং স্টেট (যেমন স্কেলিটন বা স্পিনার) দেখায়, আর ভেতরে React Suspense ব্যবহার করে কন্টেন্ট স্ট্রিম করে পাঠায় — পুরো পেজ সাদা রেখে অপেক্ষা করাতে হয় না।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

রেস্টুরেন্টে অর্ডার দেওয়ার পর ওয়েটার যেমন বলে 'স্যার, আপনার খাবার তৈরি হচ্ছে, এই নিন পানি আর স্যালাড ততক্ষণে' — খালি টেবিলে বসিয়ে না রেখে কিছু একটা দিয়ে রাখা। loading.js ঠিক এই কাজটাই করে, ইউজারকে ফাঁকা স্ক্রিনে বসিয়ে না রেখে তাৎক্ষণিক প্রতিক্রিয়া দেখায়।

📍 কোথায় ব্যবহার করবেন (Where to Use)

যেসব পেজে ডেটাবেস বা এক্সটার্নাল API থেকে ডেটা আনতে কিছুটা সময় লাগে (ড্যাশবোর্ড, প্রোডাক্ট লিস্ট) সেখানে ইউজার-এক্সপেরিয়েন্স ভালো রাখতে।

```
JavaScript / JSX

// app/dashboard/loading.js
// dashboard ডেটা লোড না হওয়া পর্যন্ত এই UI দেখা যাবে

export default function Loading() {
  return <p>ড্যাশবোর্ড লোড হচ্ছে...</p>;
}
```

■ সংজ্ঞা (Definition)

error.js ফাইল একটি React Error Boundary হিসেবে কাজ করে। কোনো রুট সেগমেন্টে রানটাইমে এরর হলে, পুরো অ্যাপ ত্র্যাশ না করে শুধু সেই অংশটুকু error.js এ ডিফাইন করা UI দিয়ে বদলে যায় — সাথে একটি reset ফাংশন দেওয়া হয় যা দিয়ে আবার চেষ্টা করা যায়।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

একটা বহুতল ভবনের একটা রুমে আগুন লাগলে যদি পুরো বিল্ডিং ভেঙে ফেলতে হতো, সেটা ভয়ংকর হতো। কিন্তু ফায়ার-প্রুফ দরজা (error.js) থাকলে শুধু ওই রুমটা বন্ধ হয়ে যায়, বাকি ভবনের মানুষ স্বাভাবিকভাবে চলাফেরা করতে পারে।

📍 কোথায় ব্যবহার করবেন (Where to Use)

যেকোনো প্রোডাকশন অ্যাপে — যেখানে একটা কম্পোনেন্টের এরর যাতে পুরো ওয়েবসাইট ভেঙে না দেয়, এবং ইউজারকে একটা বন্ধুত্বপূর্ণ এরর মেসেজ + রিট্রাই অপশন দেখাতে চান।

```
JavaScript / JSX

// app/dashboard/error.js
'use client'; // Error Boundary অবশ্যই Client Component হতে হবে

export default function Error({ error, reset }) {
  return (
    <div>
      <h2>কিছু একটা সমস্যা হয়েছে!</h2>
      <button onClick={() => reset()}>আবার চেষ্টা করুন</button>
    </div>
  );
}
```

পর্ব ৪ — কম্পোনেন্ট ও ডেটা

৪.১ সার্ভার কম্পোনেন্ট বনাম ক্লায়েন্ট কম্পোনেন্ট

Server vs Client Components

■ সংজ্ঞা (Definition)

App Router-এ ডিফল্টভাবে প্রতিটা কম্পোনেন্ট Server Component — এগুলো সার্ভারে রেন্ডার হয়ে HTML হিসেবে ব্রাউজারে পাঠানো হয়, তাই এতে ব্রাউজারের JavaScript বাডলে জায়গা নেয় না। যেসব কম্পোনেন্টে ইন্টারঅ্যাক্টিভিটি (useState, onClick, useEffect) দরকার, সেগুলোর ফাইলের একদম উপরে 'use client' লিখে Client Component বানাতে হয়।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

সার্ভার কম্পোনেন্ট হলো রান্নাঘরে বাবুর্চির রান্না করা খাবার — অতিথি (ব্রাউজার) শুধু প্লেটে সাজানো খাবার (HTML) পায়, রান্নাঘরের হাঁড়িপাতিল (JavaScript কোড) তার কাছে পাঠাতে হয় না। আর ক্লায়েন্ট কম্পোনেন্ট হলো টেবিলে রাখা DIY সালাদ বার — অতিথিকে নিজে হাত দিয়ে (ইন্টারঅ্যাকশন) কিছু একটা বানিয়ে নিতে হবে, তাই তার হাতে দরকারি টুল (JS) দিতেই হবে।

📍 কোথায় ব্যবহার করবেন (Where to Use)

স্ট্যাটিক কন্টেন্ট, ডেটা ফেচিং, SEO-নির্ভর পেজের জন্য Server Component; আর ফর্ম, মডাল, কাউন্টার, ড্রাগ-ড্রপের মতো ইন্টারঅ্যাক্টিভ UI-এর জন্য Client Component ব্যবহার করবেন।

```
JavaScript / JSX

// app/counter.js
'use client'; // ইন্টারঅ্যাক্টিভ, তাই Client Component
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      ক্লিক হয়েছে: {count} বার
    </button>
  );
}
```

Server Component

(ডিফল্ট আচরণ)

- ✓ সার্ভারে রেন্ডার হয়
- ✓ ব্রাউজারে JS পাঠায় না
- ✓ সরাসরি ডেটাবেস/API কল করা যায়
- ✗ useState / onClick নেই

```
export default function Page() { ... }
```

Client Component

('use client' প্রয়োজন)

- ✓ ব্রাউজারে রেন্ডার/রান হয়
- ✓ useState, useEffect কাজ করে
- ✓ onClick, onChange ইভেন্ট
- ✗ সরাসরি সার্ভার রিসোর্স নেই

```
'use client';
```

চিত্র: সার্ভার কম্পোনেন্ট বনাম ক্লায়েন্ট কম্পোনেন্ট — ভিজুয়াল উপস্থাপন

■ সংজ্ঞা (Definition)

Server Component-এর ভেতরে সরাসরি `async/await` দিয়ে ডেটাবেস বা API থেকে ডেটা আনা যায় — আলাদা করে `useEffect` বা লোডিং স্টেট ম্যানেজ করার দরকার নেই, কারণ পুরো কম্পোনেন্টটাই সার্ভারে রেন্ডার হওয়ার আগে ডেটা রেডি হয়ে যায়। Next.js নিজে থেকেই `fetch()` কল ক্যাশ ও ডিডুল্লিকেট করে।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

এটা রেস্টুরেন্টের ওয়েটারের বদলে সরাসরি বাবুর্চির বাজার থেকে সবজি (ডেটা) নিজে এনে রান্না করার মতো — মাঝখানে অতিরিক্ত কাউকে (ক্লায়েন্ট-সাইড লোডিং স্টেট, এক্সট্রা রাউন্ড-ট্রিপ) দিয়ে আনা-নেওয়া করতে হচ্ছে না, তাই দ্রুত ও সহজ।

📍 কোথায় ব্যবহার করবেন (Where to Use)

ব্লগ পোস্টের লিস্ট, প্রোডাক্ট ক্যাটালগ, ইউজার প্রোফাইল ডেটা — অর্থাৎ যেকোনো পেজ যেখানে রেন্ডার হওয়ার আগে সার্ভার থেকে ডেটা দরকার।

```
JavaScript / JSX

// app/posts/page.js
async function getPosts() {
  const res = await fetch('https://api.example.com/posts', {
    next: { revalidate: 3600 }, // ১ ঘন্টা পরপর রিভ্যালিডেট
  });
  return res.json();
}

export default async function PostsPage() {
  const posts = await getPosts();
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

■ সংজ্ঞা (Definition)

Server Action হলো এমন একটি async ফাংশন যা সরাসরি সার্ভারে এক্সিকিউট হয়, কিন্তু কল করা যায় ক্লায়েন্ট থেকে ফর্ম সাবমিট বা বাটন ক্লিকের মাধ্যমে — আলাদা করে API রুট বানানোর দরকার হয় না। ফাংশনের উপরে 'use server' লিখে এটি চিহ্নিত করা হয়।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

এটা অনেকটা ব্যাংকের ডিপোজিট স্লিপ জমা দেওয়ার মতো — আপনি (ক্লায়েন্ট) শুধু ফর্ম পূরণ করে জমা দেন, ব্যাংকের ভেতরের কর্মচারী (সার্ভার) সেটা প্রসেস করে টাকা জমা করে। আপনাকে নিজে থেকে ব্যাংকের ভল্টে (ডেটাবেসে) ঢুকে টাকা রাখার সিস্টেম বানাতে হয় না।

🎯 কোথায় ব্যবহার করবেন (Where to Use)

ফর্ম সাবমিশন (কমেন্ট পোস্ট করা, প্রোডাক্ট অর্ডার করা, প্রোফাইল আপডেট) — যেকোনো জায়গায় ইউজারের ইনপুট নিয়ে সার্ভারে ডেটা পরিবর্তন (create/update/delete) করতে হবে।

```
JavaScript / JSX

// app/actions.js
'use server';

export async function createPost(formData) {
  const title = formData.get('title');
  await db.posts.create({ title });
}

// app/new-post/page.js
import { createPost } from '../actions';

export default function NewPost() {
  return (
    <form action={createPost}>
      <input name="title" placeholder="পোস্টের শিরোনাম" />
      <button type="submit">পাবলিশ করুন</button>
    </form>
  );
}
```

 সংজ্ঞা (Definition)

app/ ফোল্ডারে route.js ফাইল দিয়ে কাস্টম API এন্ডপয়েন্ট বানানো যায়। এতে GET, POST, PUT, DELETE-এর মতো HTTP মেথড অনুযায়ী আলাদা আলাদা ফাংশন এক্সপোর্ট করা হয় — এটা মূলত একটা ব্যাকএন্ড API সার্ভারের কাজ করে।

 বাস্তব জীবনের উপমা (Real-life Analogy)

এটা একটা রেস্টুরেন্টের অর্ডার-কাউন্টারের মতো, যেখানে বাইরের যেকোনো অ্যাপ (মোবাইল অ্যাপ, থার্ড-পার্টি সার্ভিস) এসে খাবারের অর্ডার (রিকোয়েস্ট) দিতে পারে এবং একটা নির্দিষ্ট ফরম্যাটে উত্তর (JSON রেসপন্স) পায় — সরাসরি রান্নাঘরে ঢুকতে হয় না।

 কোথায় ব্যবহার করবেন (Where to Use)

যখন আপনার ডেটা বা ফাংশনালিটি একটি মোবাইল অ্যাপ, থার্ড-পার্টি সার্ভিস (যেমন পেমেন্ট গেটওয়ের ওয়েবহুক), বা কোনো external client-কে JSON আকারে সার্ভ করতে হবে।

```
JavaScript / JSX

// app/api/posts/route.js
import { NextResponse } from 'next/server';

export async function GET() {
  const posts = await db.posts.findAll();
  return NextResponse.json(posts);
}

export async function POST(request) {
  const data = await request.json();
  const newPost = await db.posts.create(data);
  return NextResponse.json(newPost, { status: 201 });
}
```

পর্ব ৫ — অপ্টিমাইজেশন

৫.১ মিডলওয়্যার

Middleware

📖 সংজ্ঞা (Definition)

middleware.js প্রজেক্টের রুটে রাখা একটি ফাইল, যা প্রতিটি রিকোয়েস্ট সার্ভারে পৌঁছানোর আগেই এক্সিকিউট হয়। এটি দিয়ে রিকোয়েস্ট পরীক্ষা করে রিডাইরেস্ট, রিরাইট, হেডার পরিবর্তন বা অথেন্টিকেশন চেক করা যায়।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

বিমানবন্দরের ইমিগ্রেশন কাউন্টারের মতো — যাত্রী (রিকোয়েস্ট) দেশে ঢোকার (সার্ভারে পৌঁছানোর) আগেই তার পাসপোর্ট চেক করা হয়, প্রয়োজনে অন্য লাইনে পাঠানো হয় (রিডাইরেস্ট) বা প্রবেশ আটকে দেওয়া হয় (ব্লক)।

📍 কোথায় ব্যবহার করবেন (Where to Use)

লগইন-প্রোটেক্টেড পেজে অননুমোদিত ইউজারকে লগইন পেজে পাঠানো, A/B টেস্টিং, ভাষা/দেশ অনুযায়ী রিডাইরেস্ট, বট ব্লক করা ইত্যাদি কাজে।

```
JavaScript / JSX

// middleware.js (প্রজেক্টের রুটে)
import { NextResponse } from 'next/server';

export function middleware(request) {
  const isLoggedIn = request.cookies.get('token');

  if (!isLoggedIn && request.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.redirect(new URL('/login', request.url));
  }
  return NextResponse.next();
}

export const config = {
  matcher: '/dashboard/:path*',
};
```

■ সংজ্ঞা (Definition)

next/image কম্পোনেন্ট স্বয়ংক্রিয়ভাবে ছবি রিসাইজ, ফরম্যাট কনভার্ট (WebP) ও লেজি-লোড করে, ফলে পেজ দ্রুত লোড হয়। একইভাবে next/font ব্যবহার করলে গুগল ফন্ট বিন্ড-টাইমে ডাউনলোড হয়ে সেলফ-হোস্ট হয়ে যায়, ফলে এক্সটার্নাল নেটওয়ার্ক রিকোয়েস্ট লাগে না ও লেআউট শিফট কমে।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

সাধারণ ট্যাগ হলো বাজার থেকে কাঁচা সবজি না ধুয়ে সরাসরি টেবিলে দেওয়ার মতো — সাইজ, মান নিয়ন্ত্রণ নেই। আর next/image হলো একজন শেফ যিনি প্রতিটা উপকরণ সঠিক সাইজে কেটে, পরিষ্কার করে, প্রয়োজন অনুযায়ী পরিবেশন করেন।

📍 কোথায় ব্যবহার করবেন (Where to Use)

ওয়েবসাইটে যেকোনো ছবি দেখানোর সময় (প্রোডাক্ট ছবি, ব্যানার, প্রোফাইল পিকচার) এবং গুগল ফন্ট ব্যবহার করার সময়।

```
import Image from 'next/image';
import { Hind_Siliguri } from 'next/font/google';

const hind = Hind_Siliguri({ subsets: ['bengali'], weight: '400' });

export default function Banner() {
  return (
    <div className={hind.className}>
      <Image
        src="/banner.jpg"
        alt="ব্যানার"
        width={800}
        height={400}
        priority
      />
    </div>
  );
}
```

■ সংজ্ঞা (Definition)

প্রতিটি `page.js` বা `layout.js` ফাইলে একটি `metadata` অবজেক্ট অথবা `generateMetadata` ফাংশন এক্সপোর্ট করে টাইটেল, বর্ণনা, ওপেন-গ্রাফ ছবি ইত্যাদি সেট করা যায় — Next.js এগুলো স্বয়ংক্রিয়ভাবে HTML-এর `<head>` ট্যাগে বসিয়ে দেয়।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

এটা একটা বইয়ের প্রচ্ছদ ও ব্লার্বের মতো — বইয়ের ভেতরের কন্টেন্ট যত ভালোই হোক, প্রচ্ছদে সঠিক নাম-পরিচয় (`metadata`) না থাকলে লাইব্রেরিতে (সার্চ ইঞ্জিনে) কেউ বইটা খুঁজে পাবে না বা চিনবে না।

📍 কোথায় ব্যবহার করবেন (Where to Use)

প্রতিটি পাবলিক পেজে — বিশেষ করে ব্লগ, প্রোডাক্ট পেজ, ল্যান্ডিং পেজে — যেখানে গুগল সার্চ বা সোশ্যাল মিডিয়া শেয়ারে সঠিকভাবে দেখানো জরুরি।

```
JavaScript / JSX

// app/blog/[slug]/page.js
export async function generateMetadata({ params }) {
  const post = await getPost(params.slug);
  return {
    title: post.title,
    description: post.excerpt,
    openGraph: {
      images: [post.coverImage],
    },
  };
}

export default function BlogPost({ params }) {
  // ...
}
```

পর্ব ৬ — রেন্ডারিং ও প্রোডাকশন

৬.১ স্ট্যাটিক বনাম ডাইনামিক রেন্ডারিং

Static vs Dynamic Rendering

📖 সংজ্ঞা (Definition)

স্ট্যাটিক রেন্ডারিং (SSG)-এ পেজের HTML বিল্ড টাইমে একবার তৈরি হয়ে থাকে এবং সব ইউজারের জন্য একই ফাইল সার্ভ হয় — খুব দ্রুত ও CDN-friendly। ডাইনামিক রেন্ডারিং-এ প্রতিটি রিকোয়েস্টে নতুন করে HTML তৈরি হয়, যা পার্সোনালাইজড বা রিয়েল-টাইম ডেটার জন্য দরকার।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

স্ট্যাটিক রেন্ডারিং হলো প্রিন্ট করা পত্রিকার মতো — একবার ছাপা হলে সবাই একই কপি পড়ে। ডাইনামিক রেন্ডারিং হলো প্রতিটা কাস্টমারের জন্য আলাদা করে তৈরি করা কাস্টম অর্ডার (যেমন ব্যক্তিগত রেস্টুরেন্ট বিল) — প্রতিবার নতুন করে হিসাব কষে বানাতে হয়।

📍 কোথায় ব্যবহার করবেন (Where to Use)

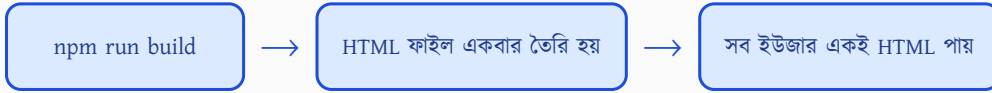
ব্লগ, মার্কেটিং পেজ, ডকুমেন্টেশন সাইটের জন্য স্ট্যাটিক রেন্ডারিং; আর ইউজার-স্পেসিফিক ড্যাশবোর্ড, লাইভ স্টক প্রাইস, পার্সোনালাইজড ফিডের জন্য ডাইনামিক রেন্ডারিং ব্যবহার করবেন।

```
JavaScript / JSX

// স্ট্যাটিক রেন্ডারিং (ডিফল্ট আচরণ)
export default async function Page() {
  const data = await fetch('https://api.example.com/data');
  // বিল্ড টাইমে একবার রেন্ডার হবে
  return <div>{/* ... */}</div>;
}

// ডাইনামিক রেন্ডারিং বাধ্যতামূলক করতে
export const dynamic = 'force-dynamic';
```

● স্ট্যাটিক রেন্ডারিং (Build Time)



● ডাইনামিক রেন্ডারিং (Request Time)



চিত্র: স্ট্যাটিক বনাম ডাইনামিক রেন্ডারিং — ভিজুয়াল উপস্থাপন

৬.২ এনভায়রনমেন্ট ভেরিয়েবল

Environment Variables

■ সংজ্ঞা (Definition)

.env.local ফাইলে সিক্রেট কী, ডেটাবেস URL ইত্যাদি রাখা হয়। যেসব ভেরিয়েবলের নামের শুরুতে NEXT_PUBLIC_ থাকে, শুধু সেগুলোই ব্রাউজার (ক্লায়েন্ট) সাইডে অ্যাক্সেসযোগ্য হয় — বাকি সব শুধু সার্ভারে থাকে, ফলে গোপন তথ্য নিরাপদ থাকে।

💡 বাস্তব জীবনের উপমা (Real-life Analogy)

এটা অফিসের ফাইল ক্যাবিনেটের মতো — কিছু ফাইল (NEXT_PUBLIC_) রিসেপশনে রাখা থাকে যা যে কেউ দেখতে পারে, আর কিছু ফাইল (সাধারণ env variable) ম্যানেজারের লকড ড্রয়ারে থাকে যা শুধু ভেতরের স্টাফ (সার্ভার) দেখতে পায়।

📍 কোথায় ব্যবহার করবেন (Where to Use)

API কী, ডেটাবেস পাসওয়ার্ড, থার্ড-পার্টি সিক্রেট টোকেন সংরক্ষণ করতে এবং ডেভেলপমেন্ট বনাম প্রোডাকশনে ভিন্ন ভিন্ন কনফিগারেশন রাখতে।

```
# .env.local
DATABASE_URL=postgres://user:pass@host/db      # শুধু সার্ভারে অ্যাক্সেসযোগ্য
NEXT_PUBLIC_ANALYTICS_ID=UA-12345678          # ব্রাউজারেও অ্যাক্সেসযোগ্য

# কোডে ব্যবহার
# const dbUrl = process.env.DATABASE_URL;
```

 সংজ্ঞা (Definition)

Next.js অ্যাপ Vercel-এ (যারা Next.js তৈরি করেছে) এক-ক্লিকে ডেপ্লয় করা যায় — গিট রিপোজিটরি কানেক্ট করলেই স্বয়ংক্রিয়ভাবে বিল্ড ও হোস্টিং হয়ে যায়। এছাড়া `npm run build` চালিয়ে নিজস্ব Node.js সার্ভার, Docker কন্টেইনার, বা অন্য যেকোনো ক্লাউড প্ল্যাটফর্মেও ডেপ্লয় করা সম্ভব।

 বাস্তব জীবনের উপমা (Real-life Analogy)

এটা একটা রেস্টোরাঁর ফ্র্যাঞ্চাইজি খোলার মতো — Vercel হলো একটা রেডিমেড শপিং মলের দোকান ভাড়া নেওয়া, যেখানে বিদ্যুৎ-পানি-নিরাপত্তা সব আগে থেকেই ব্যবস্থা করা (এক ক্লিকে ডেপ্লয়)। আর নিজের সার্ভারে ডেপ্লয় করা হলো নিজের জমিতে নিজে দোকান বানানো — বেশি নিয়ন্ত্রণ, কিন্তু বেশি কাজও।

 কোথায় ব্যবহার করবেন (Where to Use)

ডেভেলপমেন্ট শেষ হলে আপকে বিশ্বব্যাপী ইউজারদের জন্য লাইভ করার সময়। ছোট প্রজেক্ট বা স্টার্টআপের জন্য Vercel সবচেয়ে সহজ পথ।

Terminal

প্রোডাকশন বিল্ড তৈরি করা

`npm run build`

লোকালি প্রোডাকশন মোডে চালানো

`npm run start`

অথবা Vercel CLI দিয়ে সরাসরি ডেপ্লয়

`npx vercel --prod`



আপনি Next.js এর মূল কনসেপ্টগুলো ধাপে ধাপে সম্পন্ন করেছেন।

এখন নিজে হাতে একটা ছোট প্রজেক্ট বানিয়ে প্র্যাকটিস করুন —

যেমন একটা ব্যক্তিগত ব্লগ বা টু-ডু লিস্ট অ্যাপ।

পরবর্তী ধাপ: Next.js অফিসিয়াল ডকুমেন্টেশন (nextjs.org/docs) থেকে

Parallel Routes, Intercepting Routes ও Caching নিয়ে আরও গভীরে যান।